

GSR-GNN: Training Acceleration and Memory-Saving Framework of Deep GNNs on Circuit Graph

Yuebo Luo

luo00466@umn.edu

University of Minnesota, Twin Cities
Minneapolis, Minnesota, USA

Shiyang Li

li004074@umn.edu

University of Minnesota, Twin Cities
Minneapolis, Minnesota, USA

Yifei Feng

yf3005@nyu.edu

New York University
Brooklyn, New York, USA

Vishal Kancharla

kanch042@umn.edu

University of Minnesota, Twin Cities
Minneapolis, Minnesota, USA

Shaoyi Huang

shuang59@stevens.edu

Stevens Institute of Technology
Hoboken, New Jersey, USA

Caiwen Ding

dingc@umn.edu

University of Minnesota, Twin Cities
Minneapolis, Minnesota, USA

Abstract

Graph neural networks (GNNs) show strong promise for circuit analysis, but scaling to modern large-scale circuit graphs is limited by GPU memory and training cost, especially for deep models. We revisit deep GNNs for circuit graphs and show that, when trainable, they significantly outperform shallow architectures, motivating an efficient, domain-specific training framework. We propose Grouped-Sparse-Reversible GNN (GSR-GNN), which enables training GNNs with up to hundreds of layers while reducing both compute and memory overhead. GSR-GNN integrates reversible residual modules with a group-wise sparse nonlinear operator that compresses node embeddings without sacrificing task-relevant information, and employs an optimized execution pipeline to eliminate fragmented activation storage and reduce data movement. On sampled circuit graphs, GSR-GNN achieves up to 87.2% peak memory reduction and over 30× training speedup with negligible degradation in correlation-based quality metrics, making deep GNNs practical for large-scale EDA workloads.

Keywords

GNN, EDA, Circuit Design, Reversible Computing

1 Introduction

Graph Neural Networks (GNNs) have shown strong and consistent performance in circuit graph analysis [14, 26]. For example, verifying a Booth multiplier can take over 100 hours using the commercial OneSpin equivalence checking tool [23], while GNNs can finish the same task in less than one minute.

With ongoing technological advances, circuit graphs, graph data extracted from circuit designs, are becoming significantly larger and more complex [5, 6, 12, 13, 24] with greatly varied connection patterns. This growth has pushed traditional shallow GNNs—typically 3–4 layers—to their performance limits [10, 28].

Prior research has paved the way for training extremely deep GNNs, reaching hundreds or even thousands of layers. RevGNN [18], for instance, introduces a group-wise reversible residual structure. It significantly reduces the memory complexity of GNNs from $O(LND)$ (where L is depth, N is nodes, D is features) to $O(ND)$, effectively making it independent of depth. Our observations in Figure 1 show that training deep RevGNN leads to significant performance improvements in downstream circuit design tasks compared

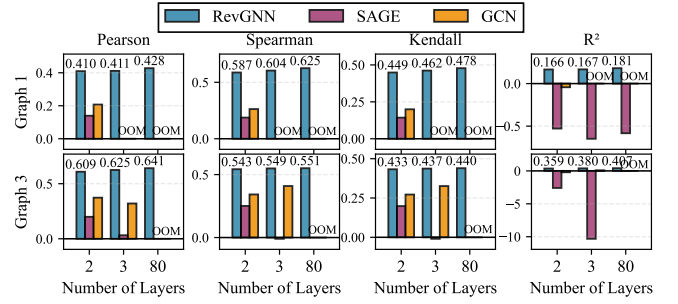


Figure 1: Performance Comparison: RevGNN vs. SAGE vs. GCN on Circuit Graphs 1 and 3 (See Table 1), where RevGNN outperforms SAGE and GCN and keeps gaining return when increasing depth. Negative scores in Spearman, Kendall, and R^2 mean failure to find correlations.

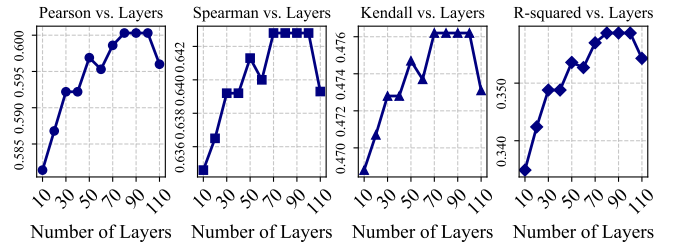


Figure 2: Training results of baseline RevGNN on example graph 356-R in Table 1, where RevGNN plateaus around 70 layers, where SAGE and GCN are either too low to stay in the range or crashed because of out-of-memory (OOM) error.

to conventionally shallow GNN architectures, where RevGNN’s Pearson can be pushed to 0.4277 at 80 layers from 0.4096 at two layers, whereas GCN [15] and GraphSAGE [11] stays below 0.3731 at best or even zero scores when there are two layers.

Nonetheless, when approaching its theoretical potential in circuit design domains, deep RevGNN hits a scalability bottleneck rooted in its chunk-based, create-then-destroy memory management strategy. Our profiling of RevGNN on the CircuitNet dataset [13] reveals that this memory management approach creates a cascade of practical limitations: (i) excessive GPU memory reservations (e.g., 17910

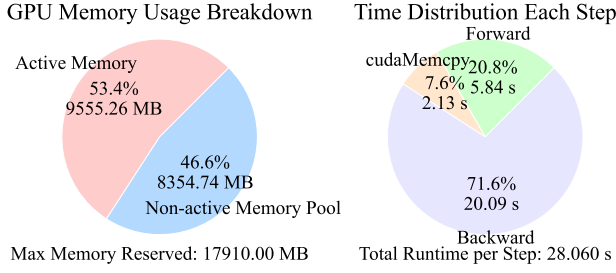


Figure 3: GPU memory usage and epoch runtime summary of 100-layer RevGNN with 384 hidden channels on Graph 356-RISCY-a1-c5 in Table 1.

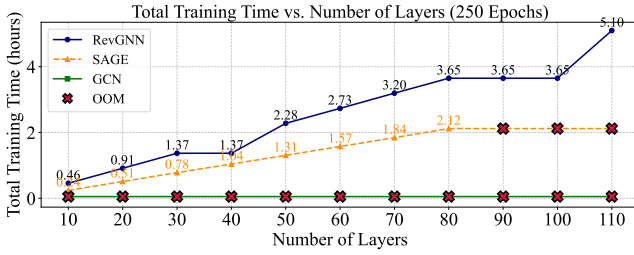


Figure 4: Example training time of baseline 100-layer RevGNN on graph 7 in Table 1. Note that the red 'X' refers to an out-of-memory (OOM) error.

MB reserved with only 9555.26 MB active, as shown in Figure 3), prohibitively long training times (5.10 hours for 250 epochs on a single circuit graph 356-R selected from Table 1, shown in Figure 4), and eventual performance degradation in all four correlation scores: Pearson, Spearman, Kendall, and R^2 beyond optimal depth at around 70 layers (please see Figure 2). Collectively, these issues make RevGNN practically difficult for real-world circuit congestion prediction tasks. It requires up to 46.4 GPU hours and up to 30+ GB of GPU memory to train a 100-layer, 384-hidden-channel model on just 10 sampled graphs, while the entire dataset has 10,370 different circuit graphs in total.

To overcome these challenges, we streamline RevGNN’s memory management approach, introduce efficient embedding compression with the relative sparse message passing carried by its GNN blocks, and eventually, propose our contributions to the solution as follows:

- To boost training efficiency, we introduce Grouped-Sparse-Reversible GNN (GSR-GNN), a high-speed and memory-efficient domain-specific deep GNN framework. GSR-GNN leverages a reversible residual structure to enable deep GNN training. We further accelerate training by incorporating embedding compression during both forward and backward passes, without negatively impacting model accuracy.
- By optimizing the memory management workflow for deep GNN training, we’ve achieved substantial improvements in memory efficiency, maintaining over 90% memory utilization and occupying much lower overall memory usage. Our framework also strikes an excellent balance between memory usage and training speed.

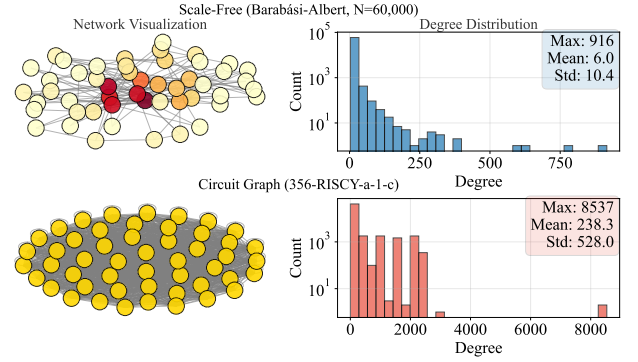


Figure 5: Comparison of Conventional graph dataset (upper) and circuit graph 4 (lower), Table 1 in 50-node graph component pattern and node degree distribution. Circuit graph shows an apparently different connecting pattern and node degree distribution from conventional graph.

- We profile our framework against the domain-specific circuit graph dataset CircuitNet. The evaluation results highlight our framework’s bottleneck training speed and memory-saving capabilities in deep GNN training for circuit graph datasets compared to baselines.

Experiment results demonstrate that GSR-GNN achieves up to above 35× speedup, GPU memory saving of 87.22%, and 98.6% of reserved memory is active over RevGNN on sampled circuit graph datasets when performing congestion prediction tasks [2, 16, 27, 29].

2 Background

Graph Learning in Circuit Design: Circuit design workflows generate complex circuit representations ideal for graph-based learning [20, 22]. In contrast to a universal graph dataset such as the Barabási-Albert [3], shown in Figure 5, circuit graphs exhibit unique characteristics: massive nodes, more bi-extreme towards either a few (as few as one) or many neighbours (more than 8,000) per node. Almost even and regular connectivity patterns in the microscope, along with diverse feature distributions, challenge conventional GNN implementations in this domain.

Recent efforts have produced comprehensive circuit datasets [5, 13] containing thousands of designs for tasks such as congestion prediction [8, 16], Design Rules Check (DRC), timing estimation, and placement optimization, enabling the realistic evaluation of GNN efficiency improvements in terms of computation speedup and end-to-end performance.

Reversible GNNs: On top of the residual connection architecture to further the depth of GCN [7, 17, 19], RevGNN [18] addresses a fundamental memory bottleneck in training deep GNNs. Traditional deep GNNs suffer from prohibitive memory overhead that scales linearly with the number of layers, as intermediate activations must be stored for backpropagation. This limits the practical depth of GNNs. RevGNN introduces a reversible architecture [9, 18], where the input to each layer can be reconstructed from its output during the backward pass. By eliminating the need to store intermediate activations, RevGNN makes memory complexity independent of network depth, significantly reducing the memory overhead during

training. This breakthrough enables training GNNs with hundreds of layers on a single GPU.

3 Problem Statement and Challenges

In this section, we first introduce the necessary terminology and notations used in the paper. Then, we present our observations that challenge current RevGNNs. Given graph $G = (V, E)$ with adjacency matrix $A \in \mathbb{R}^{N \times N}$, node embeddings at layer l are denoted as $X^l \in \mathbb{R}^{N \times D}$, where $N = |V|$ and D is the feature dimension. Network depth is L with layer index $l \in \{1, 2, \dots, L\}$, producing final predictions $\hat{y} = f_\theta(X^L) \in \mathbb{R}^N$. During backpropagation, gradients flow as $\frac{\partial \mathcal{L}}{\partial y} \rightarrow \frac{\partial \mathcal{L}}{\partial X^L} \rightarrow \dots \rightarrow \frac{\partial \mathcal{L}}{\partial X^1}$. The Split operation partitions tensors along the feature dimension, followed by Conventional GNN operations that process full-dimensional node embeddings and gradients:

$$X_1^l, X_2^l = \text{Split}(X^l) \quad \text{where } X_1^l, X_2^l \in \mathbb{R}^{N \times D/2} \quad (1)$$

$$X^{l+1} = \text{GNN-Block-Forward}(G, X^l) \quad (2)$$

$$\frac{\partial \mathcal{L}}{\partial X^l} = \text{GNN-Block-Backward}(G^T, \frac{\partial \mathcal{L}}{\partial X^{l+1}}) \quad (3)$$

where forward pass aggregates neighbor information over G , and backward pass propagates gradients through G^T .

3.1 Challenges for Reversible GNNs

Despite theoretical advantages, reversible GNN face significant practical limitations that hinder its adoption in real-world applications. We identify three critical challenges that motivate the need for more efficient alternatives.

Prohibitive Time-for-Space Tradeoff: RevGNN adopts dynamic memory management by partitioning input features $X \in \mathbb{R}^{N \times D}$ into C groups, where $N = |V|$ is the number of vertices and D is the hidden dimension. This approach employs a *create-then-destroy* cycle, in terms of detach, copy and then delete at each layer $t \in [1, L]$ that incurs significant overhead. During forward propagation, activations are immediately deallocated after use:

$$\mathcal{M}_{\text{input}}^{(t)} \leftarrow \emptyset \quad (\text{deallocate via storage.resize(0)}), \quad \text{Peak}(\mathcal{M}) = O\left(\frac{N \cdot D}{C}\right) \quad (4)$$

During backward propagation, the destroyed inputs must be reconstructed via memory reallocation and invertible computation:

$$\mathcal{M}_{\text{input}}^{(t)} \leftarrow \text{resize}\left(\prod_i d_i\right) = \text{malloc}(N \cdot D \cdot \text{sizeof(float32)}), \quad X_{\text{input}}^{(t)} \leftarrow f^{-1}(Y^{(t)}) \quad (5)$$

The frequent allocation-deallocation cycle introduces three sources of overhead: (1) system calls for memory management, (2) memory fragmentation reducing cache efficiency, and (3) the invertible computation itself. The grouped reversible architecture computes:

$$\text{Forward: } Y'_0 = \sum_{j=2}^C X_j, \quad Y'_i = f_{\theta_i}(Y'_{i-1}, A) + X_i, \quad i \in \{1, \dots, C\} \quad (6)$$

$$\text{Inverse: } X_i = Y'_i - f_{\theta_i}(Y'_{i-1}, A), \quad Y'_{i-1} = \begin{cases} Y'_{i-1} & i > 1 \\ \sum_{j=2}^C X_j & i = 1 \end{cases} \quad (7)$$

where each partition $X_i \in \mathbb{R}^{N \times D/C}$ and f_{θ_i} is a GNN block. The inversion requires reverse iteration through all C groups. Aggregating over L layers, without considering the edge embedding, the total computational complexity is:

$$C_{\text{total}} = O\left(L \cdot \left(N \cdot \frac{D^2}{C}\right)\right) + \Theta(L) \quad (8)$$

where $\Theta(L)$ represents the cumulative allocation overhead. The timing analysis reveals a severe imbalance, with the total epoch

time decomposed as:

$$T_{\text{epoch}} = T_{\text{forward}} + T_{\text{copy}} + T_{\text{backward}} \quad (9)$$

Empirical Analysis: On a representative EDA benchmark, forward propagation requires $T_{\text{forward}} = 5.84\text{s}$ (20.9%), memory copy (cudaMemcpy) takes $T_{\text{copy}} = 2.13\text{s}$ (7.6%), while invertible backpropagation with repeated reallocation dominates at $T_{\text{backward}} = 20.02\text{s}$ (71.6%), yielding $T_{\text{epoch}} = 27.99\text{s}$. The backward-to-forward ratio is 3.43, and the numerical difference between T_{forward} and T_{backward} is $\Delta T \approx 14.18\text{s}$ per epoch. The backward-to-copy ratio of 9.40 further highlights the computational burden from the create-destroy mechanism. This disproportionate cost can increase total training time by 10^1 to $10^2 \times$ for deep GNN models ($L > 50$), making RevGNN impractical for time-sensitive EDA applications (e.g. congestion prediction) where rapid design iteration is critical.

Diminishing Gain from Extreme Depth: The performance benefits of very deep RevGNNs plateau rapidly, particularly on domain-specific graphs. Figure 2 shows that correlation metrics on circuit graphs plateau around 70 layers and deteriorate beyond 100 layers. This indicates that moderate network depth achieves near-optimal performance, questioning the practical value of extreme depth capabilities while compounding the memory and computational efficiency challenges described above.

4 Architecture of GSR-GNN

In this section, we present our general model architecture design, including our custom operators utilized in GSR-GNN, optimized memory management workflow, and our algorithm design in the forward and backward phases.

4.1 Operators and Blocks used in GSR-GNN

Suppose the example graph adjacency matrix is $\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$, the conventional GNN Block of forward and backward pass [4] performs product full-dimensional node embedding or gradient processed by Rectified Linear Units (ReLU [1]) $\text{ReLU}(X^l)$ in $\begin{bmatrix} 0.37 & 0.76 & 0 & 0.79 \\ 0.58 & 0 & 0.57 & 0 \\ 0 & 0.55 & 0.95 & 0 \\ 0 & 0 & 0.23 & 0 \end{bmatrix}$

and $\text{ReLU}(\frac{dX^{l+1}}{dL})$ in $\begin{bmatrix} 0.23 & 0 & 0.89 & 0 \\ 0.45 & 0.78 & 0 & 0.67 \\ 0 & 0 & 0 & 0.88 \\ 0 & 0.33 & 0.66 & 0 \end{bmatrix}$, resulting in $\begin{bmatrix} 0.37 & 1.31 & 0.95 & 0.79 \\ 0.58 & 0 & 0.8 & 0 \\ 0.37 & 0.76 & 0.23 & 0.79 \\ 0.58 & 0.55 & 1.52 & 0 \end{bmatrix}$ and

$\begin{bmatrix} 0.23 & 0 & 0.89 & 0.38 \\ 0.56 & 1.11 & 0.66 & 0.67 \\ 0.23 & 0.33 & 1.55 & 0 \\ 0.45 & 1.11 & 0 & 1.05 \end{bmatrix}$ as X^{l+1} and $\frac{dX^l}{dL}$ respectively. In contrast, our GSR-GNN Block introduces Group Sparse GS(X, k) $\rightarrow (V, I)$ as a counterpart to ReLU. Where values $V \in \mathbb{R}^{N \times k}$ and indices $I \in \mathbb{Z}^{N \times k}$ are equivalent to sparsifying the node embedding with pre-defined k

values, which is Val: $\begin{bmatrix} 0.79 \\ 0.58 \\ 0.95 \\ 0.86 \end{bmatrix}$ and Idx: $\begin{bmatrix} 3 \\ 0 \\ 2 \\ 3 \end{bmatrix}$ for forward block example, where each row of val is the K-most significant elements selected from full-dimensional node embedding. And Val: $\begin{bmatrix} 0.12 \\ 0.45 \\ 0.55 \\ 0.99 \end{bmatrix}$ for the backward block example while keeping the same val saved since forward.

The indices I/idx record the "activated" neurons' positions to source V/val in the embedding from the forward pass. It is preserved and used in the backward process to source the gradient that shares the same positions as in the embedding; this step is an indispensable contributing factor to performance. Then, our sparse message

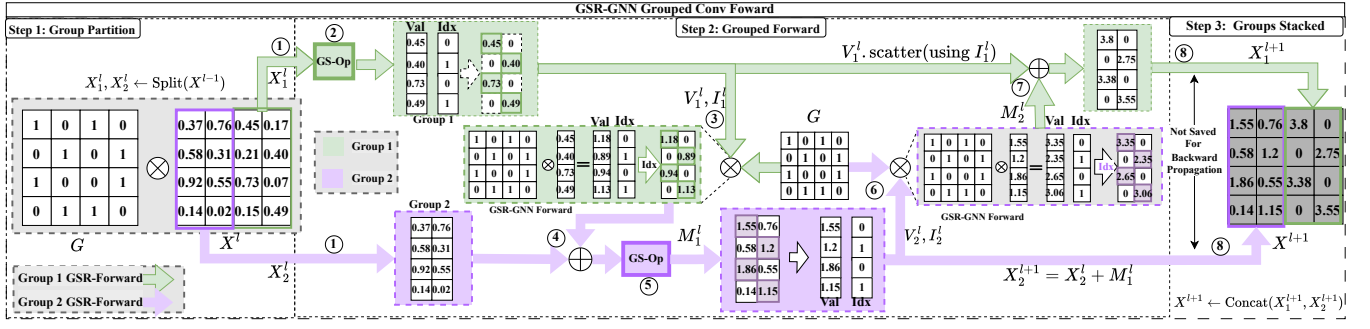


Figure 6: GSR-GNN in-layer forward workflow.

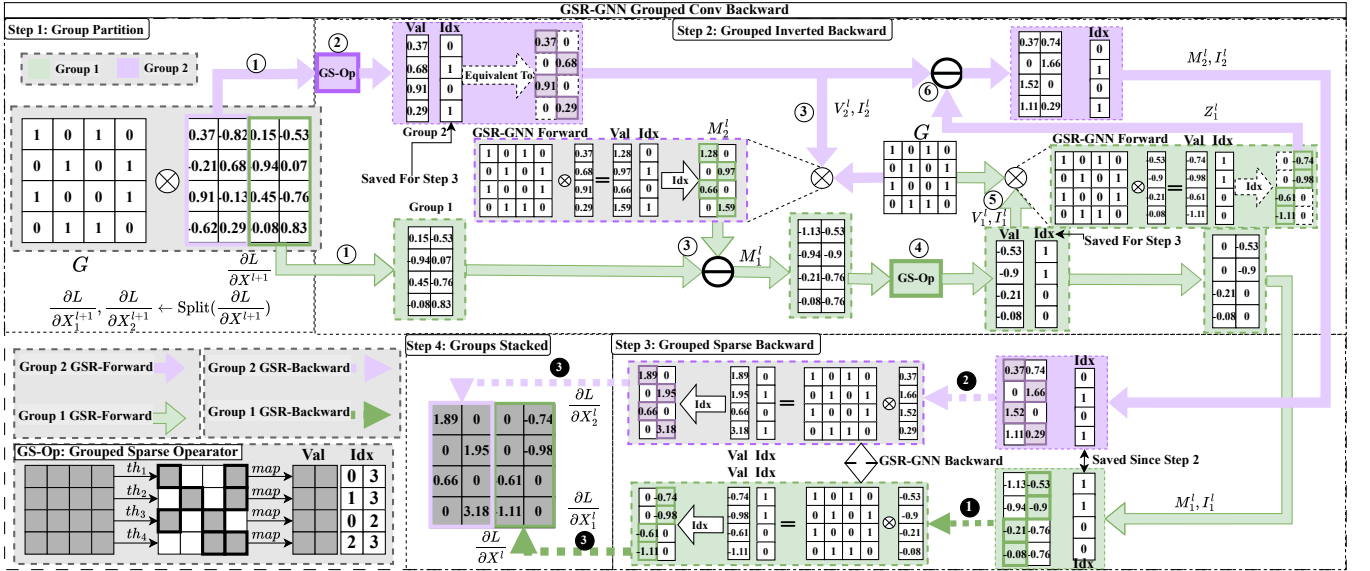


Figure 7: GSR-GNN in-layer backward workflow.

passing occurs via:

$$X^{l+1} = \text{GSR-GNN-Forward-Block}(V^l, I^l, G) \quad (10)$$

$$\frac{\partial \mathcal{L}}{\partial X^l} = \text{GSR-GNN-Backward-Block}(M^l, I^l, G) \quad (11)$$

Then, X^{l+1} is $\begin{bmatrix} 0 & 0.95 & 0.79 \\ 0.58 & 0 & 0.86 \\ 0 & 0 & 1.65 \\ 0.58 & 0.95 & 0 \end{bmatrix}$ and $\frac{\partial \mathcal{L}}{\partial X^l}$ becomes $\begin{bmatrix} 0 & 0 & 0.67 & 0 \\ 1.44 & 0 & 0 & 0 \\ 0 & 0 & 1.11 & 0 \\ 0 & 0 & 0 & 1.0 \end{bmatrix}$ through product and scatter. With (V, I) , our forward block promptly uses values in the node embedding indexed by the indices while skipping zero elements, rather than accessing full-dimensional embeddings per node when it comes to the partially reversible backward phase. Similarly, the GSR-GNN backward block also efficiently utilizes I generated in the forward block to record neurons' positions for reference, so the corresponding backward block only reads the gradients whose positions are recorded by I to compute and then finish one learning step.

Algorithm 1 GSR-GNN Forward

```

1: Input: Graph  $G$ , features  $X^0$ , layers  $L$ , GS parameter  $k$ 
2: Output: Predictions  $\hat{y}$ 
3: for  $l = 1$  to  $L$  do
4:    $X_1^l, X_2^l \leftarrow \text{Split}(X^{l-1})$ ;
5:    $V_1^l, I_1^l \leftarrow \text{GS}(X_1^l, k)$  ▷ Group partition & GS
6:    $M_1^l \leftarrow \text{GSR-Forward}(V_1^l, I_1^l, G)$ 
7:    $X_1^{l+1} \leftarrow X_2^l + M_1^l$ 
8:    $V_2^l, I_2^l \leftarrow \text{GS}(X_2^l, k)$ 
9:    $M_2^l \leftarrow \text{GSR-Forward}(V_2^l, I_2^l, G)$ 
10:   $X_1^{l+1} \leftarrow \text{scatter}(V_1^l, I_1^l) + M_2^l$ 
11:   $X^{l+1} \leftarrow \text{Concat}(X_1^{l+1}, X_2^{l+1})$  ▷ Stack groups
12: end for
13: return  $\text{OutputLayer}(X^L)$ 

```

4.2 Enhanced Memory Management

Our GSR-GNN framework changes the in-group workflow compared to conventional RevGNN. Unlike RevGNN's frequent detach-copy-delete cycles for both node embeddings (forward pass) and gradient tensors (backward pass), our custom blocks eliminate these costly memory operations.

Algorithm 2 GSR-GNN Backward

```

1: Input: Gradient  $\partial L / \partial \hat{y}$    Output: Gradient  $\partial L / \partial X^0$ 
2:  $g^L \leftarrow \text{OutputLayerBackward}(\partial L / \partial \hat{y})$ 
3: for  $l = L$  down to 1 do
4:    $(g_1^{l+1}, g_2^{l+1}) \leftarrow \text{Split}(g^{l+1}); (V_2, I_2) \leftarrow \text{GS}(g_2^{l+1}, k)$     $\triangleright$  Group partition & GS
5:    $M_1 \leftarrow g_1^{l+1} - \text{GSRBlock}(V_2, I_2, G)$ 
6:    $(V_1, I_1) \leftarrow \text{GS}(M_1, k); Z_1 \leftarrow \text{GSRBlock}(V_1, I_1, G)$ 
7:    $M_2 \leftarrow V_2.\text{scatter}(I_2) - Z_1$ 
8:    $g_1^l \leftarrow \text{GSRBlockBackward}(M_1, I_1, G); g_2^l \leftarrow \text{GSRBlockBackward}(M_2, I_2, G)$ 
9:    $g^l \leftarrow \text{Concat}(g_1^l, g_2^l)$ 
10: end for
11: Return  $g^0$ 

```

This optimization yields three key benefits: (1) **reduced fragmentation** by avoiding unnecessary tensor copies, (2) **improved memory utilization** through elimination of inactive memory pools, and (3) **selective save-for-backward** operations that only store indices when required for subsequent backward blocks.

While our approach maintains $O(LNK)$ memory complexity linear in the number of layers L , the actual memory footprint remains significantly lower than RevGNN within the practical depth range. This design ensures predictable memory usage while maximizing GPU memory efficiency for deep graph neural networks.

4.3 Forward Phase of GSR-GNN

Figure 6 and 7 provide an overview of GSR-GNN. For each layer, during the forward propagation shown in Figure 6, the input node embedding is partitioned evenly into two groups, split

$\begin{bmatrix} 0.37 & 0.76 \\ 0.58 & 0.31 \\ 0.92 & 0.55 \\ 0.14 & 0.02 \end{bmatrix}$ and $\begin{bmatrix} 0.45 & 0.17 \\ 0.21 & 0.40 \\ 0.73 & 0.07 \\ 0.15 & 0.49 \end{bmatrix}$. This is step ① from line 5, Algorithm 1. Group

1 $\begin{bmatrix} 0.45 & 0.17 \\ 0.21 & 0.40 \\ 0.73 & 0.07 \\ 0.15 & 0.49 \end{bmatrix}$ firstly proceeds through the GS-Op at ② (line 5, Algorithm 1) to produce the compressed CBSR-format Val:

$\begin{bmatrix} 0.45 \\ 0.40 \\ 0.73 \\ 0.49 \end{bmatrix}$, idx:

$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}$, which become the input of the GSR-GNN forward block first

(③, line 6 of Algorithm 1), then the result $\begin{bmatrix} 1.18 & 0 \\ 0 & 0.89 \\ 0.94 & 0 \\ 0 & 1.13 \end{bmatrix}$ added to Group

2 $\begin{bmatrix} 0.45 & 0.17 \\ 0.21 & 0.40 \\ 0.73 & 0.07 \\ 0.15 & 0.49 \end{bmatrix}$, which subsequently enters the second GS-Op at ⑤ at

line 8 and produces Val: $\begin{bmatrix} 1.55 \\ 1.2 \\ 1.86 \\ 1.15 \end{bmatrix}$ and Idx: $\begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \end{bmatrix}$, then another forward

block at ⑥, line 9. Following this, the output of the second forward block $\begin{bmatrix} 3.35 & 0 \\ 0 & 2.35 \\ 2.65 & 0 \\ 0 & 3.06 \end{bmatrix}$ is also added back to group 1 at ⑦ to be the updated

channel 1, shown by line 10, 11 of Algorithm 1, meanwhile becomes the updated channel 2 of the node embedding at ⑧ with the updated group 1, $X_1^{l+1} = \begin{bmatrix} 3.8 & 0 \\ 0 & 2.75 \\ 3.38 & 0 \\ 0 & 3.55 \end{bmatrix}$, this also updates channel 2 of the node embedding to be passed to the next layer.

4.4 Backward Phase of GSR-GNN

During the backward propagation in Figure 7 and Algorithm 2, the gradient passed from the next layer is split into two groups

Table 1: Circuit Graph Dataset Summary (Ordered by Size)

Size	#	Name	Node	Edge	Train/Val/Test Labels	Range
L	1	5324-F	69.8k	22.5M	48.8/10.5/10.5k	131 [0.0, 1.0]
	2	4474-F	65.3k	21.8M	52.3/6.5/6.5k	255 [0.0, 1.04]
M	3	876-R	49.6k	12.2M	39.7/5.0/5.0k	60 [0.0, 0.22]
	4	1522-R	49.5k	12.2M	39.6/5.0/5.0k	106 [0.0, 0.36]
	5	1886-R	49.1k	11.7M	39.3/4.9/4.9k	134 [0.0, 0.58]
	6	639-R	49.1k	11.7M	34.3/7.4/7.4k	105 [0.0, 0.48]
	7	356-R	49.0k	11.7M	39.2/4.9/4.9k	271 [0.0, 0.90]
S	8	6859-F	44.3k	2.7M	35.4/4.4/4.4k	28 [0.0, 1.0]
	9	6227-F	43.4k	2.7M	34.7/4.3/4.3k	148 [0.0, 3.75]
	10	7157-R	38.7k	6.8M	30.9/3.9/3.9k	153 [0.0, 0.66]

Note: "F" and "R" in the column Name refer to "FPU" and "RISC-Y".

split $\begin{bmatrix} 0.37 & -0.82 & 0.15 & -0.53 \\ -0.21 & 0.68 & -0.94 & 0.07 \\ 0.91 & -0.13 & 0.45 & -0.76 \\ -0.62 & 0.29 & -0.08 & 0.83 \end{bmatrix} = \begin{bmatrix} 0.37 & -0.82 \\ -0.21 & 0.68 \\ 0.91 & -0.13 \\ -0.62 & 0.29 \end{bmatrix}, \begin{bmatrix} 0.15 & -0.53 \\ -0.94 & 0.07 \\ 0.45 & -0.76 \\ -0.08 & 0.83 \end{bmatrix}$ (step ①, line 4, Al-

gorithm 2), whereas, at this time, group 2 $\begin{bmatrix} 0.37 & -0.82 \\ -0.21 & 0.68 \\ 0.91 & -0.13 \\ -0.62 & 0.29 \end{bmatrix}$ goes through

GS-Op to output CBSR values $\begin{bmatrix} 0.37 \\ 0.68 \\ 0.91 \\ 0.29 \end{bmatrix}$ and indices $\begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$ at ②, and

enter the GSR-GNN forward block at ③ at line 5 in Algorithm 2 and note that positions (indices) of the activated neurons are saved for the step at line 8. Then, the output of the first forward block

$\begin{bmatrix} 1.28 & 0 \\ 0 & 0.97 \\ 0.66 & 0 \\ 0 & 1.59 \end{bmatrix}$ goes to the subtraction with group 1 (line 7, Alg 2), the

result $\begin{bmatrix} 1.13 & 0.53 \\ 0.94 & 0.9 \\ 0.21 & 0.76 \\ 0.08 & 0.76 \end{bmatrix}$ goes into two ways: one enters another GS-Op at

④ with the result $\begin{bmatrix} -0.53 \\ -0.9 \\ -0.21 \\ -0.08 \end{bmatrix}$ and indices $\begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, which enter the second

forward block at ⑤, whose result $\begin{bmatrix} 0 & -0.74 \\ 0 & -0.98 \\ -0.61 & 0 \\ -1.11 & 0 \end{bmatrix}$ performs subtraction

with group 2 at ⑥ to be the input of its GSR-GNN backward block at ①; the other is kept for the dedicated backward block at ②. This process is covered by lines 8 in Algorithm 2. Using the remembered positions from the previous forward-style recomputation,

both channels of the gradient $\frac{\partial L}{\partial X_1^l} \begin{bmatrix} 1.89 & 0 \\ 0 & 1.95 \\ 0.66 & 0 \\ 0 & 3.18 \end{bmatrix}$ and $\frac{\partial L}{\partial X_2^l} \begin{bmatrix} 0 & -0.74 \\ 0 & -0.98 \\ -0.61 & 0 \\ -1.11 & 0 \end{bmatrix}$ are

updated to be passed to the last layer at ③.

5 Experiments

This section presents our experimental setup and evaluates the performance of GSR-GNN against baseline methods. We demonstrate the framework's advantages in terms of training acceleration and memory efficiency while maintaining comparable accuracy.

Platform: All experiments were conducted on an AMD EPYC 7763 64-Core Processor server (504GB RAM) and a single NVIDIA A6000-48GB GPU (CUDA toolkit 12.6), PyG version 2.5.0, Pytorch 2.1.2.

Dataset: We evaluate our approach using the CircuitNet dataset [5], which originates from over 10,000 circuit graph samples serving various downstream tasks. We randomly sample 10 representative circuit designs (detailed in Table 1) following the provided pre-processing method without sub-graph partitioning to ensure design size variability and fully expose the scale of graph data. We focus primarily on congestion prediction, a critical step in the circuit design process [5, 13, 22]. Following standard practice in circuit design applications [25], we evaluate performance using three rank

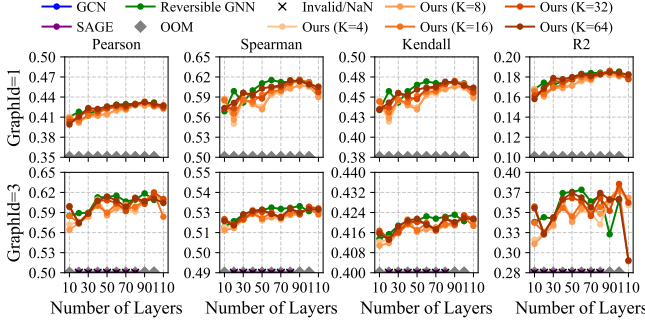


Figure 8: Correlation performance of GSR-GNN over GCN/SAGE/RevGNN in selected graphs (Graph 1 and Graph 3 in Table 1). The results are averaged over hidden dimension=256 and 384. The higher correlation scores mean better performance, non-reversible baselines (SAGE, GCN) are out-of-range due to very low scores or out-of-memory errors.

correlation metrics: Pearson, Kendall, and Spearman correlation scores, with higher values indicating better performance.

Models and Configuration: We compared our model with state-of-the-art GraphSAGE [21], GCN [15], and the official RevGNN [18]. The hyperparameters include: hidden dimension, varying between 256 and 384; the number of layers, ranging from 10 to 100 with a step of ten layers each; While ReLU from baselines doesn’t need parameter choices, the K-values for our GS-Op fall in the set {4, 8, 16, 32, 64}, the training takes up 250 epochs, the train, validation and test set division are given in Table 1.

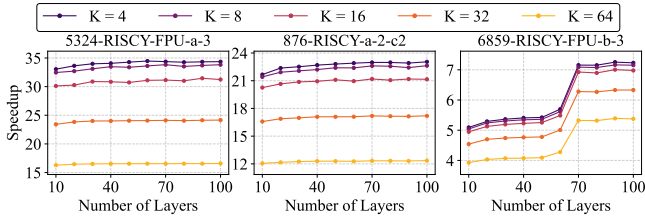


Figure 9: Speedup of GSR-GNN over Reversible GNNs at varying sparsity levels defined by K, the greater the K, the more elements preserved per row of activations and lower sparsity.

5.1 Experimental Results

This section demonstrates that GSR-GNN achieves training acceleration while maintaining comparable correlation performance to standard reversible deep GNN models. Given that non-reversible GCN and SAGE models exhibit severe performance fluctuations with increasing depth and remain inferior to RevGNN (Figure 1), our primary comparison focuses on RevGNN.

Figure 8 evaluates GSR-GNN with GS-Op ($K \in \{4, 8, 16, 32, 64\}$) against RevGNN on CircuitNet for congestion prediction across 10 randomly sampled graphs in Table 1. The correlation metrics demonstrate that GSR-GNN maintains performance comparable to RevGNN, with mean differences of +0.0026 (Pearson), +0.0067 (Spearman), +0.0053 (Kendall), and -0.023 (R^2). These negligible

Table 2: Runtime Distribution: 100-layer, 384-hidden channels Baseline (RevGNN) vs Ours (Graph 3 in Table 1)

Method	Forward↓	Backward↓	cudaMemCpy↓	Total/Spdup↑
Baseline	5.84s (20.8%)	20.09s (71.6%)	2.13s (7.6%)	28.06s
Ours (K=4)	0.36s (23.7%)	1.16s (76.2%)	0.001s (0.1%)	1.52s (18.4×)
Ours (K=8)	0.37s (24.2%)	1.17s (75.7%)	0.001s (0.1%)	1.54s (18.2×)
Ours (K=16)	0.40s (24.1%)	1.25s (75.9%)	0.001s (0.1%)	1.64s (17.1×)
Ours (K=32)	0.50s (24.1%)	1.59s (75.9%)	0.001s (0.1%)	2.09s (13.4×)
Ours (K=64)	0.76s (24.7%)	2.31s (75.2%)	0.001s (0.1%)	3.07s (9.1×)

differences indicate that GS-Op preserves model accuracy while enabling the observed speedups.

Figure 9 illustrates the end-to-end training speedup of GSR-GNN at two hidden channels, 256 and 384, over RevGNN across three circuit scales. GSR-GNN achieves over 35× (K=4, 384 hidden channels) for graph 5324-RISCY-FPU-a-3, over 4× (K=64) for graph 6859-RISCY-FPU-b-3, with performance inversely correlated to sparsity level. In addition, we revisit the breakdown of training step runtime since Figure 3, comparing our GSR-GNN against RevGNN on the same graph dataset 356-RISCY-a1-c5.

Table 2 shows that GSR-GNN has obvious acceleration across three sections: Forward, Backward, and cudaMemCpy. The most significant training component, Backward, is reduced to 1.52 seconds at best, and forward is 0.36 seconds. Our memory management optimization consistently cuts cudaMemCpy time to 0.0001 seconds during the whole training workflow, making a final best speedup of 18.4×. Though increasing sparsity in terms of K lowers the relative speedup effect, the worst end-to-end speedup can still reach 9.1×.

Furthermore, Figure 10 presents the memory efficiency of GSR-GNN compared to RevGNN on graph 876-R and 5324-R from Table 1. Due to optimized memory management and compressed node embeddings, within the optimal range of the number of layers, GSR-GNN consistently consumes less GPU memory than RevGNN, even at 100 layers. GSR-GNN comes with a clear, minimum gap of on average 2.58GB and 5.043GB, respectively, for both graphs. Although the memory savings decrease with increasing layers, GSR-GNN consistently maintains over 90% memory utilization, whereas RevGNN always exhibits approximately 50% memory waste. This shows that our GSR-GNN reduces the maximum number of GNNs that can be trained on circuit graph datasets of various sizes. Additionally, this enables training of circuit graph datasets on GPUs with less than 8 GB of memory at layers less than 30. To be closer to baseline performance while maintaining relative speedup, one can also increase both layers and lower the sparsity to achieve near-optimal performance in correlation metrics.

6 Conclusion

In our paper, we propose a GNN training framework targeting faster and more GPU memory-efficient deep GNN training on circuit graph datasets. We redesign the grouped convolution and reversible residual module of RevGNN with custom grouped sparse reversible GNN blocks and applied it to graph datasets sampled from CircuitNet in varied, representative sizes. In the evaluation on target circuit graph datasets, our framework achieves apparent training acceleration effect (over 35× at most), lower GPU memory

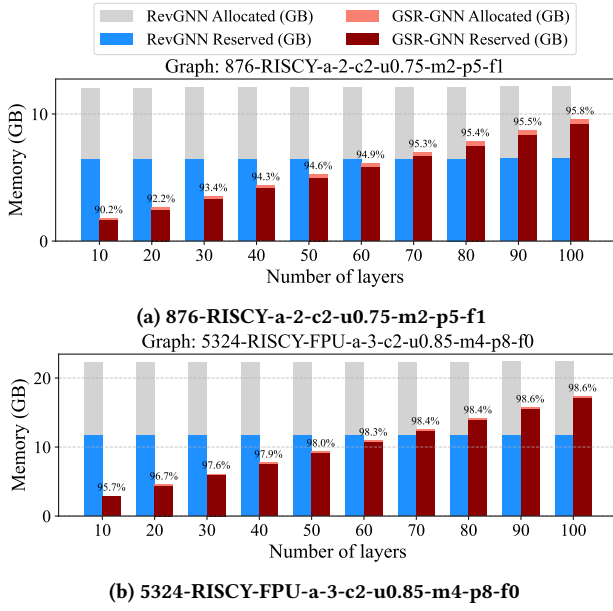


Figure 10: Memory efficiency comparison between RevGNN and GSR-GNN on 2 sample graphs, graph 1 and graph 3 in Table 1. The memory efficiency, expressed as a percentage, is the ratio of Allocated GPU Memory to Reserved GPU Memory during the training phase of models.

reservation (87.22% GPU memory saving at most), and higher efficiency (98.6% rate of active memory over total reserved memory), with almost unchanged performance in correlation metrics from the congestion prediction task.

Acknowledgments

This research was supported in part by NSF SHF-2505770.

References

- [1] Abien Fred Agarap. 2019. Deep Learning using Rectified Linear Units (ReLU). arXiv:1803.08375 [cs.NE] <https://arxiv.org/abs/1803.08375>
- [2] Mohamed Baker Alawieh, Wuxi Li, Yibo Lin, Love Singhal, Mahesh A Iyer, and David Z Pan. 2020. High-definition routing congestion prediction for large-scale FPGAs. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 26–31.
- [3] Réka Albert and Albert-László Barabási. 2002. Statistical mechanics of complex networks. *Reviews of Modern Physics* 74, 1 (Jan. 2002), 47–97. doi:10.1103/revmodphys.74.47
- [4] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. 2019. Deep Equilibrium Models. arXiv:1909.01377 [cs.LG] <https://arxiv.org/abs/1909.01377>
- [5] Zhuomin Chai, Yuxiang Zhao, Wei Liu, Yibo Lin, Runsheng Wang, and Ru Huang. 2023. CircuitNet: An Open-Source Dataset for Machine Learning in VLSI CAD Applications With Improved Domain-Specific Evaluation Metric and Learning Strategies. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 12 (2023), 5034–5047. doi:10.1109/TCAD.2023.3287970
- [6] Animesh Basak Chowdhury, Benjamin Tan, Ramesh Karri, and Siddharth Garg. 2021. OpenABC-D: A Large-Scale Dataset For Machine Learning Guided Integrated Circuit Synthesis. arXiv:2110.11292 [cs.LG] <https://arxiv.org/abs/2110.11292>
- [7] Laurent Dinh, David Krueger, and Yoshua Bengio. 2015. NICE: Non-linear Independent Components Estimation. arXiv:1410.8516 [cs.LG] <https://arxiv.org/abs/1410.8516>
- [8] Amur Ghose, Vincent Zhang, Yingxue Zhang, Dong Li, Wulong Liu, and Mark Coates. 2021. Generalizable cross-graph embedding for GNN-based congestion prediction. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [9] Aidan N Gomez, Mengye Ren, Raquel Urtasun, and Roger B Grosse. 2017. The reversible residual network: Backpropagation without storing activations. *Advances in neural information processing systems* 30 (2017).
- [10] Zizheng Guo, Mingjie Liu, Jiaqi Gu, Shuhan Zhang, David Z. Pan, and Yibo Lin. 2022. A timing engine inspired graph neural network model for pre-routing slack prediction. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (San Francisco, California) (DAC '22)*. Association for Computing Machinery, New York, NY, USA, 1207–1212. doi:10.1145/3489517.3530597
- [11] William L. Hamilton, Rex Ying, and Jure Leskovec. 2018. Inductive Representation Learning on Large Graphs. arXiv:1706.02216 [cs.SI] <https://arxiv.org/abs/1706.02216>
- [12] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems* 33 (2020), 22118–22133.
- [13] Xun Jiang, Zhuomin Chai, Yuxiang Zhao, Yibo Lin, Runsheng Wang, and Ru Huang. 2024. CircuitNet 2.0: An Advanced Dataset for Promoting Machine Learning Innovations in Realistic Chip Design Environment. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=nMFSUjxMML>
- [14] Daniela Kaufmann. 2022. Formal verification of multiplier circuits using computer algebra. *it - Information Technology* 64, 6 (2022), 285–291. doi:10.1515/itit-2022-0039
- [15] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings*. OpenReview.net. <https://openreview.net/forum?id=SJU4ayYgl>
- [16] Robert Kirby, Saad Godil, Rajarshi Roy, and Bryan Catanzaro. 2019. Congestion-Net: Routing congestion prediction using deep graph neural networks. In *2019 IFIP/IEEE 27th International Conference on Very Large Scale Integration (VLSI-Soc)*. IEEE, 217–222.
- [17] Guohao Li, Matthias Muller, Ali Thabet, and Bernard Ghanem. 2019. DeepGCNs: Can GCNs go as deep as CNNs?. In *Proceedings of the IEEE/CVF international conference on computer vision*. 9267–9276.
- [18] Guohao Li, Matthias Müller, Bernard Ghanem, and Vladlen Koltun. 2022. Training Graph Neural Networks with 1000 Layers. arXiv:2106.07476 [cs.LG] <https://arxiv.org/abs/2106.07476>
- [19] Guohao Li, Chenxin Xiong, Ali Thabet, and Bernard Ghanem. 2020. Deepergcn: All you need to train deeper gcn. arXiv preprint arXiv:2006.07739 (2020).
- [20] Yibo Lin, Zixuan Jiang, Jiaqi Gu, Wuxi Li, Shounak Dhar, Haoxing Ren, Brucek Khailany, and David Z. Pan. 2021. DREAMPlace: Deep Learning Toolkit-Enabled GPU Acceleration for Modern VLSI Placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 40, 4 (2021), 748–761. doi:10.1109/TCAD.2020.3003843
- [21] Jielun Liu, Ghim Ping Ong, and Xiquan Chen. 2020. GraphSAGE-based traffic speed forecasting for segment network with sparse data. *IEEE Transactions on Intelligent Transportation Systems* 23, 3 (2020), 1755–1766.
- [22] Martin Rapp, Hussam Amrouch, Yibo Lin, Bei Yu, David Z. Pan, Marilyn Wolf, and Jörg Henkel. 2022. MLCAD: A Survey of Research in Machine Learning for CAD Keynote Paper. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 10 (2022), 3162–3181. doi:10.1109/TCAD.2021.3124762
- [23] Amr Sayed-Ahmed, Daniel Große, Ulrich Kühne, Mathias Soeken, and Rolf Drechsler. 2016. Formal verification of integer multipliers by combining Gröbner basis with logic reduction. In *2016 Design, Automation Test in Europe Conference Exhibition (DATE)*. 1048–1053.
- [24] Kuansan Wang, Zhihong Shen, Chiyuan Huang, Chieh-Han Wu, Yuxiao Dong, and Anshul Kanakia. 2020. Microsoft Academic Graph: When experts are not enough. *Quantitative Science Studies* 1, 1 (02 2020), 396–413. arXiv:https://direct.mit.edu/qss/article-pdf/1/1/396/1760880/qss_a_00021.pdf doi:10.1162/qss_a_00021
- [25] Shuwen Yang, Zhihao Yang, Dong Li, Yingxue Zhang, Zhanguang Zhang, Guojie Song, and Jianye HAO. 2022. Versatile Multi-stage Graph Neural Network for Circuit Representation. In *Advances in Neural Information Processing Systems*, Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho (Eds.). <https://openreview.net/forum?id=nax3ATLrovW>
- [26] Cunxi Yu, Maciej Ciesielski, and Alan Mishchenko. 2018. Fast Algebraic Rewriting Based on And-Inverter Graphs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 9 (2018), 1907–1911. doi:10.1109/TCAD.2017.2772854
- [27] Cunxi Yu and Zhiru Zhang. 2019. Painting on placement: Forecasting routing congestion using conditional generative adversarial nets. In *Proceedings of the 56th Annual Design Automation Conference 2019*. 1–6.
- [28] Wentao Zhang, Zeang Sheng, Yuezhan Jiang, Yikuan Xia, Jun Gao, Zhi Yang, and Bin Cui. 2021. Evaluating Deep Graph Neural Networks. arXiv:2108.00955 [cs.LG] <https://arxiv.org/abs/2108.00955>
- [29] Zhonghua Zhou, Ziran Zhu, Jianli Chen, Yuzhe Ma, Bei Yu, Tsung-Yi Ho, Guy Lemieux, and Andre Ivanov. 2019. Congestion-aware global routing using deep convolutional generative adversarial networks. In *2019 ACM/IEEE 1st Workshop*

on Machine Learning for CAD (MLCAD). IEEE, 1–6.